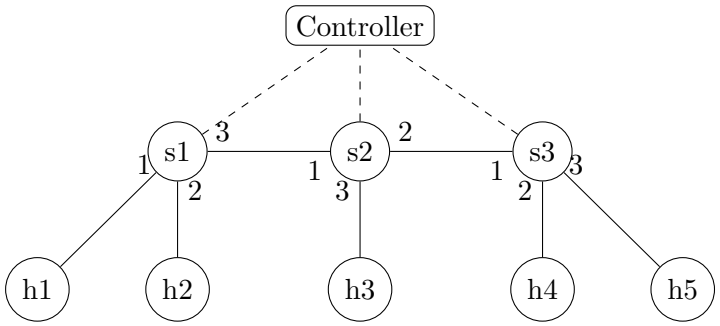
Piattaforme Software per la Rete (Mod. 1) – 13/07/2015

Su ogni foglio consegnato scrivere nome, cognome, matricola e il numero dell'esercizio.

Esercizio 1

Si consideri la rete in figura. I nodi h1,..., hn contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., hn sono MAC1,..., MACn e IP1,..., IPn. Sui nodi h1,..., hn sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse.



Il controllore Ryu è collegato ai nodi s1,..., sm ed è configurato come indicato nel Programma 1.

Scrivere tutti i pacchetti che sono inviati su tutti i collegamenti della rete e tra i nodi e il controllore in coseguenza degli eventi seguenti. Scrivere il contenuto delle tabelle openflow prima e dopo ogni domanda.

(a) Invio di un datagramma UDP da h3:12345 a h1:80 e relativa risposta.

(b) Apertura di una connessione TCP da h3:80 a h1:443.

(c) Apertura di una connessione TCP da h5:12345 a h1:80.

Programma 1 (1/2)

```
# intestazione omessa

    @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
    def switch_features_handler(self, ev):
        datapath = ev.msg.datapath
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser

        match = parser.OFPMatch()
        actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
                                          ofproto.OFPCML_NO_BUFFER)]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
                                             actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=0,
                                match=match, instructions=inst)
        datapath.send_msg(mod)

        if (datapath.id == 1) or (datapath.id == 2):
            match = parser.OFPMatch(eth_dst=MAC3)
            actions = [ parser.OFPActionOutput(3) ]
            inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
                                                 actions)]
            mod = parser.OFPFlowMod(datapath=datapath, priority=1,
                                    match=match, instructions=inst)
            datapath.send_msg(mod)

            match = parser.OFPMatch(eth_type=0x0800, # IPv4
                                    ip_proto=6,      # TCP
                                    tcp_dst=80
                                    )
            actions = [  ]
            inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
                                                 actions)]
            mod = parser.OFPFlowMod(datapath=datapath, priority=2,
                                    match=match, instructions=inst)
            datapath.send_msg(mod)
```

Programma 1 (2/2)

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
    msg = ev.msg; datapath = msg.datapath
    ofproto = datapath.ofproto; parser = datapath.ofproto_parser
    in_port = msg.match['in_port']

    pkt = packet.Packet(msg.data)
    pkt_tcp = pkt.get_protocol(tcp.tcp)

    if (pkt_tcp is not None) and (pkt_tcp.dst_port == 80):
        match = parser.OFPMatch(
            eth_type=0x0800,
            ipv4_src=pkt_ip.src,ipv4_dst=pkt_ip.dst,ip_proto=6,
            tcp_src=pkt_tcp.src_port,tcp_dst=80)
        actions = [
            parser.OFPActionSetField(tcp_dst=8080)
            parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
                                             actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=3,
                                match=match, instructions=inst)
        datapath.send_msg(mod)

        match = parser.OFPMatch(
            eth_type=0x0800,
            ipv4_src=pkt_ip.dst,ipv4_dst=pkt_ip.src,ip_proto=6,
            tcp_src=pkt_tcp.8080,tcp_dst=pkt_tcp.dst_port)
        actions = [
            parser.OFPActionSetField(tcp_src=80)
            parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
                                             actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=3,
                                match=match, instructions=inst)
        datapath.send_msg(mod)
        return

    data = msg.data
    actions = [parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
    out = parser.OFPPacketOut(datapath=datapath, buffer_id=msg.buffer_id,
                              in_port=in_port, actions=actions, data=data)
    datapath.send_msg(out)
```

Esercizio 2

Si consideri un filtro di Bloom costituito da un array di 16 bit e 3 funzioni hash. Inizialmente tutti i bit dell'array sono posti a zero.

(a) Calcolare la probabilità che un dato bit sia posto ad 1 dopo aver effettuato l'inserimento di 3 elementi

(b) Calcolare la probabilità di un falso positivo.

(c) Dire se la funzione $h(x) = 4x \bmod 16$ è una buona funzione di hash per questo filtro. Spiegare.

(d) Sono date le funzioni hash

$$h_1(x) = 3x \bmod 16$$
$$h_2(x) = 5x \bmod 16$$
$$h_3(x) = 7x \bmod 16$$

Inizialmente tutti i bit dell'array sono posti a zero. Inserire $x = 1$ e $x = 4$ nel filtro di Bloom. Mostrare lo stato del filtro dopo ogni inserimento.

(e) Conclusi gli inserimenti, verificare l'appartenza al dataset degli elementi $x = 5$ e $x = 2$. Dire se ci sono falsi positivi.

Esercizio 3

Una matrice di commutazione crossbar di uno switch 3×3 usa l'algoritmo Take-a-Ticket (TaT) per lo scheduling delle trasmissioni.

Al tempo $t = 0$ sono presenti i seguenti pacchetti, tutti di lunghezza L, in ordine dal più recente al più vecchio. Ogni pacchetto è etichettato con il numero dell'interfaccia di uscita cui è diretto.

- Ingresso 1: 1, 2, 1
- Ingresso 2: 3, 2, 1
- Ingresso 3: 2, 3, 3

(a) Svolgere l'algoritmo TaT fino allo svuotamento di tutte le code, specificando ad ogni round quali pacchetti sono trasmessi e quali ticket sono assegnati. L'algoritmo di assegnazione dei ticket privilegia sempre l'interfaccia di ingresso con l'identificativo più piccolo.

(b) Uno switch 3×3 implementa la seguente variante di TaT. Si i il numero del ticket del pacchetto che inizia la trasmissione su un'interfaccia di uscita. Se ci sono code in ingresso con numero di ticket $\geq i + 2$ per quell'interfaccia, scartano il loro pacchetto e chiedono un ticket per il pacchetto successivo.

Considerando traffico in ingresso distribuito uniformemente e code sempre piene, calcolare il numero di pacchetti scartati per ogni intervallo temporale.

(c) Si calcoli il numero di pacchetti scartati dall'algoritmo precedente nel caso di switch 4×4.

(d) Dire in cosa consiste il problema del Head-of-Line (HoL) blocking. Spiegare cosa si intende per *virtual output queues* e dire come possono affrontare il problema.